

GH Trust MFB

Bank Partner API Requirements — Wallet & Payments

For bank integration team · August 2026 · Confidential

GH Trust MFB is building a digital wallet for customers on mobile and web. We are seeking a **bank partner** to provide the payment APIs below so we can fund wallets, pay out to customers, disburse loans, and collect repayments — all under CBN MFB regulations.

What we need from you: REST APIs (JSON), sandbox environment, webhooks for real-time events, and production credentials. Our platform already integrates similar rails (Monnify, Paystack, Zest); we will build a dedicated adapter for your bank's API specification.

1. Wallet features we are building

Every feature below maps to one or more bank APIs in Section 3.

Feature	User	Description	Priority
Permanent wallet account	Customer	Each customer gets a dedicated NUBAN at signup to receive bank transfers anytime.	Phase 1
Wallet balance	Customer	Available balance and locked (pending withdrawal) balance in the app.	Phase 1
Fund wallet (bank transfer)	Customer	Customer transfers from any bank to their wallet NUBAN; balance updates automatically.	Phase 1
On-demand top-up account	Customer	Optional: temporary NUBAN for a specific amount, expiring in ~5–15 minutes (fallback / alternative channel).	Phase 2
Withdraw to bank	Customer	Customer sends wallet balance to their linked payout bank account.	Phase 1
Save payout bank account	Customer	Customer registers withdrawal account; system validates account name via NIP name enquiry.	Phase 1
Transaction history	Customer	List of credits, debits, withdrawals, and loan-related movements.	Phase 1
Loan disbursement	Admin / Ops	Send approved loan amount from MFB settlement wallet to customer's (or school's) bank account.	Phase 1

Loan processing fee	Customer	Deduct one-off fee from wallet or collect via transfer before loan proceeds.	Phase 2
Loan repayment collection	Customer / System	Collect installments via wallet debit, bank transfer to loan account, or salary direct debit.	Phase 2
Payday / salary direct debit	System	Mandate on salary account (Remita or bank-native) for automatic repayment on pay day.	Phase 2
Savings & investment debit	Customer	Fund savings plans, investments, and group contributions from wallet balance.	Phase 2
Card / USSD checkout	Customer	Alternative top-up without bank transfer (card, USSD, QR).	Phase 3
Bulk disbursement	Admin	Batch payout (e.g. multiple study loans to schools in one run).	Phase 3
Settlement wallet balance	Treasury / Admin	MFB operations view of prefunded settlement account for outbound transfers.	Phase 1

2. End-to-end flows (what your APIs must support)

2.1 Customer signup → wallet ready

1. Customer completes registration (BVN, name, phone verified).
2. We call your API to create a **reserved / dedicated NUBAN** linked to the customer.
3. Customer sees account number and bank name in the app.
4. Webhook confirms account is active (if async provisioning).

2.2 Fund wallet

1. Customer transfers NGN from any bank to their wallet NUBAN.
2. Your system sends **inbound payment webhook** with amount, reference, payer details.
3. We credit customer wallet (idempotent on your transaction reference).
4. Optional: requery API if webhook is delayed.

2.3 Withdraw to bank

1. Customer requests withdrawal; we lock the amount internally.
2. We call your **single transfer / NIP disbursement** API with bank code, account, amount, unique reference.
3. Webhook reports success, failure, or reversal.
4. We settle or release the locked balance accordingly.

2.4 Loan disbursement (admin)

1. Loan approved in admin portal with amount and customer bank details.
2. Ops initiates disbursement — same transfer API as withdrawal, different narration/metadata.
3. Success webhook → loan marked disbursed in our system.

2.5 Loan repayment (Phase 2)

1. **Option A:** Customer pays into wallet NUBAN; we allocate to loan balance internally.
2. **Option B:** Customer pays into loan-specific collection account; webhook credits loan.
3. **Option C:** Direct debit mandate debits salary account on due date.

3. Required bank APIs — complete list

Priority key: **P1** = required for wallet go-live · **P2** = loan collections & fees · **P3** = nice to have

A. Authentication & environment

A1. API authentication

P1 · OAuth2 client credentials or API key + secret with short-lived bearer token

- Token endpoint returning `access_token` and `expires_in`
- Sandbox and production base URLs
- IP allowlisting option for webhooks and API calls

A2. Merchant / MFB settlement account setup

P1 · One prefunded settlement wallet for outbound transfers

- Settlement account number provided at onboarding
- Contract / merchant ID we pass on all collection and disbursement calls

B. Customer wallet accounts (inbound collections)

B1. Create reserved / dedicated virtual account

P1 · Permanent NUBAN per customer

We send:

- `account_reference` — our unique customer ID (immutable)
- `account_name` — e.g. "GH Trust / {Customer Name}"
- `customer_email`, `customer_name`, `phone`
- `bvn` — where regulation requires BVN-linked accounts
- `currency` — NGN

You return:

- `account_number` (10-digit NUBAN)
- `bank_name`, `bank_code`
- `account_reference` (echo or your ID)
- `status` — active / pending

B2. Get reserved account details

P1 · Lookup by `account_reference`

Same fields as B1. Used for reconciliation and support.

B3. Create dynamic virtual account (optional)

P2 · Temporary NUBAN for specific amount + expiry

We send: `amount`, `currency`, `customer_email`, `transaction_reference`, `expiry_minutes`

You return: NUBAN, bank name, amount, expiry timestamp, tracking reference

B4. Deactivate / suspend virtual account

P2 · Fraud, account closure, KYC failure

C. Inbound payment notifications & verification

C1. Webhook — inbound transfer received

P1 · Real-time; HMAC-signed payload

Minimum payload fields:

- `event_type` — e.g. `payment.success`
- `transaction_reference` — your unique ID (idempotency key)
- `payment_reference` / session reference
- `amount`, `currency`, `payment_status` (PAID / FAILED)
- `account_reference` and/or `destination_account_number`
- `paid_at` timestamp
- Optional: payer bank, payer account (masked), narration

We respond HTTP 200; retries with exponential backoff on non-2xx.

C2. Verify / requery inbound transaction

P1 · GET by `transaction_reference` or `payment_reference`

Returns amount, status, account reference, timestamps. Used when webhook is missed (we poll every 2 hours).

C3. Requery virtual account payments

P2 · By account number + date range

Recovery of missed credits for a customer NUBAN.

D. Outbound transfers (withdrawals & loan disbursement)

D1. Bank account name enquiry (NIP validation)

P1 · Before saving payout account or disbursing

We send: `account_number` (10 digits), `bank_code` (NIP code)

You return: `account_name`, `account_number`, `bank_code`

D2. List supported banks

P1 · For bank picker in app

Returns bank name, NIP bank code, slug. Updated list or static file acceptable.

D3. Initiate single bank transfer

P1 · Withdrawals + loan disbursement

We send:

- `reference` — our unique idempotency key (required, max 64 chars)
- `amount` (NGN, 2 decimal places)
- `bank_code`, `account_number`, `account_name`
- `narration` — e.g. "GH Trust wallet withdrawal" / "Loan disbursement LOAN-1234"
- `currency` — NGN

You return: `reference` , `status` (pending / processing / success / failed), `transaction_id`

D4. Verify outbound transfer status

P1 · GET by `reference`

Returns final status, failure reason, session ID, completion time.

D5. Webhook — outbound transfer result

P1 · Required events:

- `transfer.success` — completed
- `transfer.failed` — failed with reason
- `transfer.reversed` — reversed after success

Payload must include our `reference` , `amount` , `status` , `transaction_id` .

D6. Bulk bank transfer

P3 · Batch loan / school payouts

Submit array of transfers; batch reference; per-item status in response/webhook.

D7. Transfer recipient registration (optional)

P2 · Pre-register beneficiary for faster repeat payouts

If your platform requires beneficiary tokens, provide create + fetch recipient APIs.

E. Treasury & settlement

E1. Settlement wallet balance

P1 · Available balance for disbursements

Returns `available_balance` , `ledger_balance` , `currency` . Used before large disbursement runs and daily treasury checks.

E2. Settlement account statement

P2 · Date-range transaction list for reconciliation

F. Direct debit & mandates (loan repayments — Phase 2)

F1. Create payment mandate

P2 · Salary account direct debit (Payday product)

We send: customer identity, bank account, max amount, frequency (monthly / on salary date), start/end date, mandate reference

You return: mandate ID, status (pending / active), authorization URL or USSD step if customer consent required

F2. Webhook — mandate activated / failed

P2

F3. Initiate direct debit collection

P2 · Debit mandate on due date

We send: mandate ID, amount, collection reference (idempotent)

You return / webhook: success, insufficient funds, mandate cancelled

F4. Cancel / suspend mandate

P2

If your bank routes mandates via Remita or NIBSS rather than a native API, provide integration guide and webhook mapping — we will adapt.

G. Alternative funding channels (Phase 3)

G1. Initialize card / USSD / QR payment

P3

Returns payment URL or USSD code; amount, reference, customer email.

G2. Webhook — card/USSD payment success

P3 · Same fields as C1 inbound webhook

H. KYC & compliance

H1. BVN-linked account provisioning

P1 · If required by CBN for dedicated accounts

Accept BVN on account creation; return verification status.

H2. Customer identity validation (optional)

P2 · BVN/NIN match before DVA activation

4. Webhook specification (summary)

Event	When	Priority
Inbound payment success	Customer funds wallet NUBAN	P1

Reserved account activated	DVA provisioning complete	P1
Reserved account failed	Provisioning rejected	P1
Outbound transfer success	Withdrawal or loan disbursement paid	P1
Outbound transfer failed	Transfer rejected	P1
Outbound transfer reversed	Bank reversal after success	P1
Mandate activated / failed	Direct debit setup	P2
Direct debit collected / failed	Repayment attempt	P2
Card/USSD payment success	Alternative top-up	P3

Webhook requirements

- HTTPS POST to our endpoint (we provide URL per environment)
- HMAC signature header (SHA-256 or SHA-512) over raw body
- Unique event / transaction ID for idempotency
- Retry policy: at least 24 hours with backoff
- Static IP list for firewall allowlisting (optional)

5. Non-functional requirements

Requirement	Expectation
Idempotency	Same reference on transfer/debit must not double-pay
Amount precision	NGN to 2 decimal places; kobo handling documented
Latency	Name enquiry < 5s; transfer initiation < 10s (async completion OK)
Availability	99.5%+ production SLA; maintenance windows communicated
Sandbox	Full API parity with test accounts, simulated webhooks, test bank codes
Documentation	OpenAPI/Swagger or PDF; error code catalogue; sample requests
Support	Technical contact for integration; dispute / failed transfer escalation path
Limits	Document per-transaction and daily limits; process to increase limits

6. Phase 1 minimum — what we need to go live

If you can only prioritise a subset, these **10 APIs + webhooks** unblock wallet and loan disbursement:

1. Authentication (A1)
2. Create reserved account (B1)
3. Get reserved account (B2)
4. Inbound payment webhook (C1)

5. Verify inbound transaction (C2)
6. Bank account name enquiry (D1)
7. List banks (D2)
8. Initiate single transfer (D3)
9. Verify outbound transfer (D4)
10. Outbound transfer webhooks (D5)
11. Settlement wallet balance (E1)

With Phase 1 only, GH Trust can: open customer wallets, accept bank transfers, process withdrawals, disburse loans to any Nigerian bank account, and reconcile automatically via webhooks and polling.

7. Phase 2 — loan collections & product debits

- Direct debit mandates (F1–F4) for Payday / salary products
- Processing fee collection from wallet or dedicated invoice
- Internal wallet debit for savings, investments, contributions (no external API — we debit ledger; bank API only needed if fee leaves settlement wallet)
- Settlement statement (E2) for finance reconciliation

8. What we build on our side (not required from bank)

- Customer wallet balance and double-entry ledger
- Withdrawal hold / release / settle logic
- Loan application and approval workflows
- Admin portal and customer mobile app
- Webhook ingestion, signature verification, idempotency store
- Repayment allocation across loan schedules (Phase 2)

9. Submission checklist for bank partner

Please provide:

- ☐ API documentation (OpenAPI preferred)
- ☐ Sandbox credentials and base URL
- ☐ Webhook signing method and sample payloads
- ☐ List of NIP bank codes supported for transfers
- ☐ Settlement account onboarding process and prefunding instructions
- ☐ Transaction and daily limit table
- ☐ Error code reference
- ☐ Direct debit / mandate availability (native or via Remita/NIBSS)
- ☐ Legal/compliance requirements for BVN-linked DVAs
- ☐ Integration timeline and dedicated technical contact

10. Contact & next steps

GH Trust will implement a bank-specific adapter against your API specification. We request a technical discovery session to walk through Phase 1 endpoints, sandbox access, and webhook setup. Once sandbox is validated, we target production wallet go-live within your onboarding timeline.